Save Point

Accounts and sync in the app

QuillFin · Prepared for Logan Young, Save Point Limited · 6 November 2026

The Flutter side of Phase 3: signing in to a QuillFin account, keeping the local database in sync with Convex, and the plan. The server side is in `backend.md`, and the design behind both is in `quillfin_plan.md` §2-3.

Two ways in

The sign-in screen offers:

- **Sign in or create an account**: a QuillFin account through Clerk. Your data syncs across devices, and you can use QuillFin's AI within your plan.
- **Continue on this device**: no account, as the app has always worked. Everything stays local.

A build without cloud config (see below) offers only the second, as does the Flutter **web** build. On the web, accounts belong to the Next.js app.

Both kinds of session survive restarts. `QuillFinAuthService` (`lib/auth/auth_service.dart`) stores the session in secure storage under `quillfin_session`.

Build config

Accounts need three publishable values at build time. Secrets never go in the app.

`--dart-define`	Value
`QUILLFIN_CLERK_DOMAIN`	Clerk's Frontend API host, e.g. `xyz.clerk.accounts.dev`
`QUILLFIN_CLERK_CLIENT_ID`	client id of the Clerk OAuth application for the app
`QUILLFIN_CONVEX_URL`	`https://<name>.convex.cloud`
`QUILLFIN_SITE_URL`	optional: the web app, for links

Copy `config/example.json` to `config/dev.json` (git-ignored), fill it in, and run:

```
flutter run --dart-define-from-file=config/dev.json
```

`CloudConfig` (`lib/config/cloud_config.dart`) reads these values. If any of the first three is missing, the build is device-only.

Signing in (OAuth 2.0 + PKCE)

`ClerkOAuth` (`lib/auth/clerk_oauth.dart`) signs in as a public OAuth client with no client secret:

1. It makes a random code verifier and state, and opens Clerk's `/oauth/authorize` page with the S256 code challenge.
2. The user signs in or signs up on Clerk's page, in the **system browser**. That's never an embedded web view (RFC 8252), so passwords and passkeys never pass through the app.
3. Clerk redirects back with a code. `BrowserAuth` (`lib/auth/browser_auth.dart`) catches the redirect:
   - **Android and iOS**: the `com.savepoint.quillfin://callback` deep link, registered in `AndroidManifest.xml` and `Info.plist` and received with `app_links`.
   - **Windows, Linux and macOS**: a loopback listener on `http://127.0.0.1:47615/callback`, open only while signing in. The port is fixed because Clerk matches redirect URIs exactly. macOS needs the `network.server` and `network.client` entitlements, which are set in both entitlement files.
4. The app checks the state, exchanges the code and verifier at `/oauth/token`, and stores the **ID token** and **refresh token**.

While the browser is open, the sign-in screen shows **Cancel**. Closing Clerk's consent screen also counts as cancelling.

The ID token is what the app sends to Convex: its audience is the OAuth client id, which `backend/convex/auth.config.ts` accepts. `idToken()` refreshes it a minute before it expires, with one refresh at a time however many calls are waiting. If Clerk refuses the refresh token (because you signed out elsewhere or it was revoked), you stay signed in on this device with your data, and sync shows "Your session has expired. Sign in again."

> Not yet checked against a real Clerk instance (no account exists yet): that Clerk's refresh grant returns a new `id_token`, and the exact redirect URI forms it accepts. The setup guide lists both as things to verify.

Local schema v3

Schema v3 (`lib/storage/db.dart`) adds three columns to each synced table (`folders`, `notes`, `questions`, `question_choices`, `question_history`, `problems`, `problem_attempts`):

Column	Meaning
`uid`	random 32-hex id shared with the server and other devices
`sync_updated_at`	last-writer-wins clock, ms since the epoch
`dirty`	1 until the row's latest change has been pushed

Triggers keep them current, not the repos, so no repo knows about sync:

- **Insert**: sets the `uid` (unless one was given), the clock, and dirty.

- **Update**: bumps the clock and sets dirty. It skips updates that set the sync columns themselves.
- **Delete**: writes a row to `sync_tombstones`. Deletes made by `ON DELETE CASCADE` fire it too, so deleting a folder tombstones every note, question, choice and history row under it.

While the sync engine applies pulled rows, it sets `sync_state.applying = 1`, which switches the triggers off. Other new tables:

- `sync_state`: the pull cursor, the account the data last synced with, and the last sync time.
- `sync_pending`: pulled rows held back until their parent arrives.

Upgrading from v2 gives every existing row a `uid` and marks it dirty, so the first sign-in uploads everything (`quillfin_plan.md` §3).

The sync engine

`SyncService` (`lib/sync/sync_service.dart`) runs one sync at a time. A request that arrives during a run queues exactly one more run after it.

1. `bindUser`: if this device last synced with a *different* account, every row is marked dirty and the cursor is reset. The local copy merges into the new account.
2. `users:ensure` (once per session).
3. **Push**: `SyncStore.nextPushBatch` collects up to 200 dirty rows, parents first, then tombstones, mapped to the server shape by `lib/sync/sync_tables.dart` (camelCase keys, booleans, parent `uid`s). After `sync:push`, `markPushed` clears `dirty` only on rows whose clock hasn't moved since they were read, so an edit made mid-push goes up next time. Rows the server skipped as older are cleared too; the pull brings down the newer copy.
4. **Pull**: `sync:pull` from the stored cursor, 200 rows a page, until `hasMore` is false. `applyPulled` applies each page in one transaction:
   - a local row that is still dirty and newer is kept, and wins on the next push;
   - a tombstone deletes the local row and its children;
   - a child whose parent isn't here yet goes to `sync_pending` and is retried after every page.

It runs:

- when the shell opens;
- when the app returns to the foreground;
- every 5 minutes while open;
- within about 10 s of a local change. sqflite has no change notifications, so a timer checks the partial `dirty` indexes. That's slower than the plan's 2 s debounce, but needs no hooks in the repos.

When a pull changes local rows, the Today, Notes and Tracker tabs refresh.

Notes over 200 KB (the server's limit) stay on this device instead of failing every push, and Settings > Sync says how many.

Screens

- **Settings > Sync**: last sync, changes waiting, the last error (with a reassurance when offline), oversized notes, and **Sync now**.
- **Settings > Plan**: Free or Pro, this month's use of QuillFin's AI, renewal or end date, **Upgrade monthly / yearly** (Stripe Checkout) or **Manage plan** (Billing Portal), opened in the browser. The plan reloads when you come back to the app. The iOS and Android store builds only show the plan: both stores require their own billing for subscriptions and forbid pointing elsewhere to pay (`store_listing.md`, Billing).
- **Account > Delete account**: after a confirmation, `account:deleteAccount` cancels Pro, deletes the Clerk user and every synced row, then the app deletes this device's copy and returns to sign-in.
- **Account**: the account's name and email. **Sign out** first pushes what it can, then asks whether to keep a copy on this device or delete it from this device. A device-only session gets **Sign in to sync** when the build has accounts.
- **Generate** in the note editor uses your own API key if one is set. Otherwise, when signed in, it uses QuillFin's AI (`QuillFinAIProvider`), and plan limits come back as a readable message.

The session reaches every screen through `CloudScope` (`lib/sync/cloud_session.dart`), an `InheritedNotifier` above the Navigator (in `MaterialApp.builder`), so pages pushed from the shell see it too. `RootShell` publishes the session when it starts and clears it when it closes.

Testing

- `test/sync/sync_store_test.dart`: the triggers (including cascades), the v2 to v3 migration, push batches, marking pushed, pulls with held-back parents, and account switching, on real in-memory SQLite.
- `test/sync/sync_service_test.dart`: two "devices" converging through `FakeConvex` (`test/helpers/fake_convex.dart`, an in-memory backend following the real rules), batching, errors, offline, scheduling, and `ConvexClient`.
- `test/auth/`: PKCE (including the RFC 7636 example), the token exchange, refresh, cancel, and both browser flows (a real loopback server on a test port).
- `test/screens/cloud_screens_test.dart`: sign-in, the shell starting sync, Settings > Sync and > Plan, sign-out choices, and AI on QuillFin's key.
- `test/sync/convex_live_test.dart`: the same sync against a **real** Convex deployment, skipped unless configured. See `testing.md`, "Live sync check".